Queue - Linked List Implementation

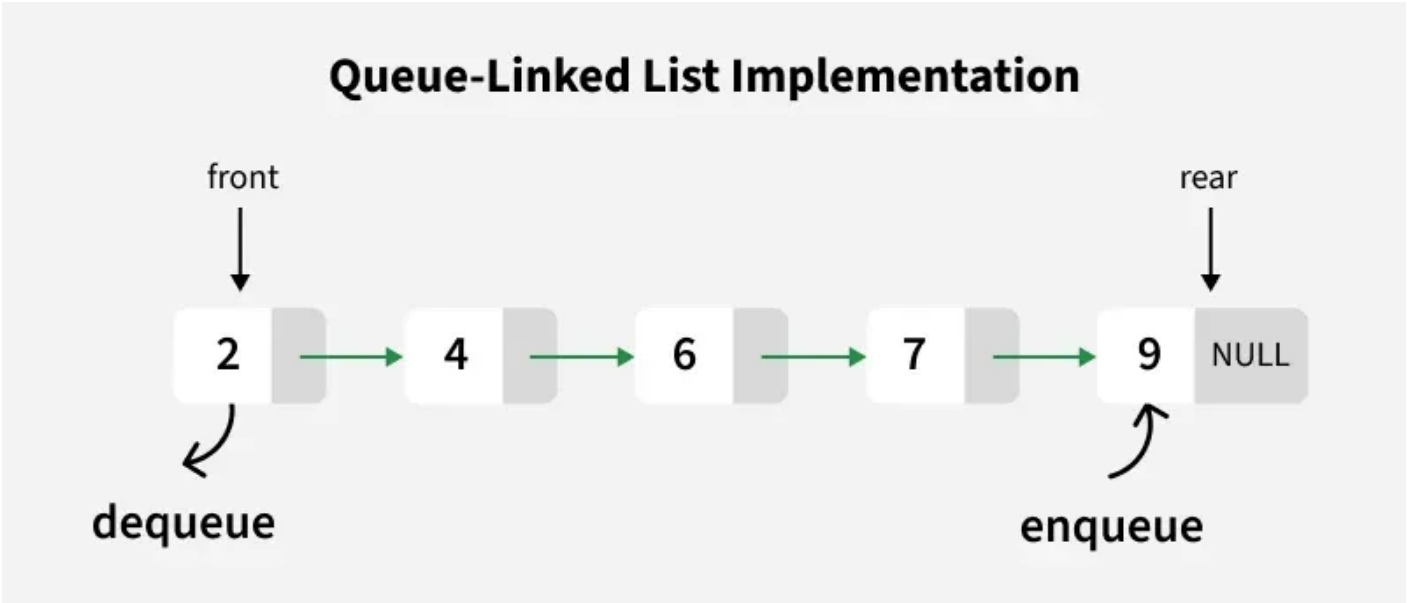
Last Updated : 25 Mar, 2025

In this article, the Linked List implementation of the queue data structure is discussed and implemented. Print '-1' if the queue is empty.

Approach: To solve the problem follow the below idea:

*we maintain two pointers, **front** and **rear**. The front points to the first item of the queue and rear points to the last item.*

- ***enQueue():** This operation adds a new node after the rear and moves the rear to the next node.*
- ***deQueue():** This operation removes the front node and moves the front to the next node.*



Queue using Linked List

Follow the below steps to solve the problem:

- Create a class Node with data members integer data and Node* next
    - A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL
- Create a class Queue with data members Node front and rear
- Enqueue Operation with parameter x:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Initialize Node* temp with data = x
- If the rear is set to NULL then set the front and rear to temp and return(Base Case)
- Else set rear next to temp and then move rear to temp

- Dequeue Operation:
  - If the front is set to NULL return(Base Case)
  - Initialize Node temp with front and set front to its next
  - If the front is equal to NULL then set the rear to NULL
  - Delete temp from the memory

Below is the Implementation of the above approach:

C++ C Java Python JavaScript

```cpp
#include <iostream>
using namespace std;

// Node class definition
class Node {
public:
    int data;
    Node* next;
    Node(int new_data) {
        data = new_data;
        next = nullptr;
    }
};

// Queue class definition
class Queue {
private:
    Node* front;
    Node* rear;
public:
    Queue() {
        front = rear = nullptr;
    }

    // Function to check if the queue is empty
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
bool isEmpty() {
    return front == nullptr;
}

// Function to add an element to the queue
void enqueue(int new_data) {
    Node* new_node = new Node(new_data);
    if (isEmpty()) {
        front = rear = new_node;
        printQueue();
        return;
    }
    rear->next = new_node;
    rear = new_node;
    printQueue();
}

// Function to remove an element from the queue
void dequeue() {
    if (isEmpty()) {
        return;
    }
    Node* temp = front;
    front = front->next;
    if (front == nullptr) rear = nullptr;
    delete temp;
    printQueue();
}

// Function to print the current state of the queue
void printQueue() {
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
        return;
    }
    Node* temp = front;
    cout << "Current Queue: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    }
};

// Driver code to test the queue implementation
int main() {
    Queue q;

    // Enqueue elements into the queue
    q.enqueue(10);
    q.enqueue(20);

    // Dequeue elements from the queue
    q.dequeue();
    q.dequeue();

    // Enqueue more elements into the queue
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);

    // Dequeue an element from the queue (this should print 30)
    q.dequeue();

    return 0;
}
```

Output

```
Current Queue: 10
Current Queue: 10 20
Current Queue: 20
Queue is empty
Current Queue: 30
Current Queue: 30 40
Current Queue: 30 40 50
Current Queue: 40 50
```

Time Complexity: O(1), The time complexity of both operations enqueue() and dequeue() is O(1) as it only changes a few pointers in both operations

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Auxiliary Space: O(1), The auxiliary Space of both operations enqueue() and dequeue() is O(1) as constant extra space is required

Related Article:

Array Implementation of Queue

Implementation of Queue using Linked List

Visit Course ↗

Comment More info Advertise with us

Next Article

Applications, Advantages and Disadvantages of Queue

Similar Reads

Queue Data Structure

A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order. It follows the principle of "First in, First out" (FIFO), where the first element adde...

2 min read

Introduction to Queue Data Structure

Queue is a linear data structure that follows FIFO (First In First Out) Principle, so the first element inserted is the first to be popped out. FIFO Principle in Queue:FIFO Principle states that the first element...

5 min read

Introduction and Array Implementation of Queue

Similar to Stack, Queue is a linear data structure that follows a particular order in which the operations are performed for storing data. The order is First In First Out (FIFO). One can imagine a queue as a line o...

2 min read

Queue - Linked List Implementation

In this article, the Linked List implementation of the queue data structure is discussed and implemented. Print '-1' if the queue is empty.Approach: To solve the problem follow the below idea:we maintain two...

8 min read

Applications, Advantages and Disadvantages of Queue

A Queue is a linear data structure. This data structure follows a particular order in which the operations are performed. The order is First In First Out (FIFO). It means that the element that is inserted first in the...

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5 min read

Different Types of Queues and its Applications

Introduction : A Queue is a linear structure that follows a particular order in which the operations are performed. The order is First In First Out (FIFO). A good example of a queue is any queue of consumers...

8 min read

Queue implementation in different languages

Some question related to Queue implementation

Easy problems on Queue

Intermediate problems on Queue

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF